翻译说明

这是一本名为 Performance Analysis and Tuning on Modern CPU 书籍的源文件存储库的中文翻译，原版由 Denis Bakhvalov 等人编写。

- 原版电子书：https://book.easyperf.net/perf_book
- 中文翻译 (第一版)：https://book.douban.com/subject/36243215/

原作者第二版正在进行中！计划的更改在谷歌文档中进行了概述。计划中的新目录在 new_toc.md 中。

目的:

- 虽然已经有翻译的书籍; 但是想 follow 更新, 借助『chatGPT』/『gemini/moonshot(kimi)』翻译成中文，(加速学习节奏，掌握，并举一反三)
- 英文源书是开源的，翻译成中文工作也持续更新，也是开源的，可以作为学习资料, 在线阅读可编辑，希望一起参与改进。
- 对每章节的内容通过『chatGPT』/『gemini/moonshot(kimi)』进行归纳总结，结巩固知识点，并对课后练习进行回答, 并验证答案。
- 最后整体勘误，定搞。

授之以鱼不如授之以渔, 使用 AI 赋能。

性能优化分析数据可以借助『chatGPT』分析。

『chatGPT』和『moonshot(kimi)』翻译效果差不多 (相同的 prompt)，但是当问文中的规划练习和代码练习时，『moonshot(kimi)』不能理解问题，不过长文本上传根据章节翻译和归纳总结不错，毕竟不用翻墙就可以使用。

[?]: 后续将上述流程用代码实现一个工作流 (尽量) 自动化翻译, 归纳,Q&A 的应用工具 (CI)。

在线阅读地址: https://weedge.github.io/perf-book-cn/zh/

中文版 PDF(推荐): https://raw.githubusercontent.com/weedge/perf-book-cn/main/perf-book-cn.pdf

学习资料

1. https://github.com/dendibakh/perf-ninja
2. https://www.youtube.com/watch?v=2tzdkC6IDbo&list=PLRWO2AL1QAV6bJAU2kgB4xfodGID43Y5d
3. 现代 cpu 微架构 (书中第一章微架构内容)：
    1. Architecture All Access: Modern CPU Architecture Part 1 – Key Concepts | Intel Technology
    2. Architecture All Access: Modern CPU Architecture 2 - Microarchitecture Deep Dive | bilibili

TIPS:

- pandoc 通过 bibtex 生成引用样式: citation-style-language(CSL)-styles, 可从 Zotero Style 官网 挑选需要样式下载
- 翻译的源文件存储库对应的 commit: 5ddfadc
- 通过`pandoc`生成 latex 源文件，具体设置见：https://github.com/weedge/perf-book/blob/feat/cn/export_book_zh.py

NOTICE:

0. 在写代码时或多或少会知道一些代码层次 (比如 c++) 的优化，但可能不知道为啥是这样的，这本书结合 cpu 讲解了相关原理 (比如`unlikely`,`likely`)。

1. 木桶效应，通过监控测量分析，寻找性能短板在哪，结合场景对症下药 (理解工作原理)
2. 了解现代 cpu 微体系架构 (本文中提到的 Intel GoldenCove 架构白皮书 [Intel, 2023b]), 以小见大, 设计思路借鉴到业务系统中 (虽然有些详细信息未公开)
3. AMD 处理器 [AMD, 2023] ; ARM Neoverse V1 处理器 [Arm, 2022b]
4. 书中一些 case 介绍了优化工具的使用和性能分析
5. 阅读本书不需要详细了解每个性能分析工具的使用 (比如常用的`perf`)，主要是针对 cpu 特性,memory 的性能分析；只要记住有这个工具干啥用的，当遇到性能分析场景时，可以再次查阅该工具的使用方法即可；主要是结合工具去实践总结方法论。
6. 本书是针对现代 CPU 的性能分析和优化；前提条件是应用程序已经消除所有主要性能问题；如果想更深层次优化 (比如底层存储系统, 流量请求最终汇聚点系统, 核心系统等)，可以使用 CPU 性能监控功能来分析和进一步调整应用程序。仅当所有高级性能问题都已修复后，才建议使用硬件功能进行低级微调。在设计不良的算法系统上进行 cpu 分析调优只是时间上的浪费。
7. 底层硬件的持续性能分析 (CP) 在 IaaS/PaaS 云服务企业中常见。
8. 延伸阅读：brendangregg-systems-performance | brendangregg-bpf-performance 必备性能分析工具书 (函方法论实践)
9. 利用人工智能和 LLM 启发架构来处理性能分析样本，分析 函数之间关系，最终高精度地找出直接影响整体吞吐量和延迟的函数和库。Raven.io提供这种功能的一家公司
10. Section 132 源代码优化章节重点掌握
    1. 编译链接层面静态分析，通过优化报告 (比如: GCC 的`-fopt-info`;clang 使用`-Rpass*`) 来获取优化建议 (需要实践测试)
    2. PGO Section 11.7 练习: https://github.com/dendibakh/perf-ninja/blob/main/labs/misc/pgo/README.md (PGO 主要用于具有大型代码库的项目，比如：数据库，分布式文件系统); 特地场景，谨慎分析配置引导优化 (可组合)。
11. 充分考虑到时间局部性和空间局部性对性能的影响
12. 尽量做扩展阅读，比如作者的博客文章，相关引用 (比如：[Fog, 2004])
13. 对于 cpu 性能优化，有些已在编译器层面进行了优化，比如机器代码布局
14. 关注低延迟系统的性能优化 Section 12.2 (比如 HFT 系统中的这个快速演讲：CppCon 2018: Jonathan Keinan "Cache Warming: Warm Up The Code"; 这些关键路径代码值钱)
15. 优化多线程应用:
    1. 借助可视化分析工具进行分析定位性能瓶颈 (同步事件，锁，上下文切换)，书中举了一些 case, Intel VTune Profiler 这个工具对于作者来说经常使用，类似 GPU 性能分析工具 nsight-compute (通过nsight-systems分析系统整体性能耗时)；
    2. 通过 eBPF 追踪内核中 futex 系统调用的执行（内核通过 futex 系统调用支持线程同步原语 - 互斥锁、信号量、条件变量等)，从涉及的线程中收集有用的元数据
    3. 尽量避免真共享下的数据竞争问题，通过编译器集成的 sanitizers 工具来识别，比如 Clang Thread sanitizer
    4. 避免伪共享。由于多核处理器 cpu 之间独立的 L1/L2 cache，会出现 cache line 不一致的问题，为了解决这个问题，有相关协议模型，常用 MESI 协议，MESI 通过这个网站模拟更直观的了解 https://www.scss.tcd.ie/Jeremy.Jones/VivioJS/caches/MESIHelp.htm ；为了保证一个 core 上修改的 cache line 数据同步到其他 core 的 cache line 上，则需要 MESI 协议来保证，如果同一个 cache line 上有个两个变量 sum1 和 sum2 之间虽然没有相互依赖逻辑，但是当修改 sum1 或者 sum2 时，需要同步同一块 cache line 的内容，导致 即使没有相互关系的变量在同一 cache line 中，需要彼此共享同步，从而出现所说的伪共享 flase sharing。伪共享因为 cache line 的同步会带来一些 cpu 时钟周期的性能损失。
    5. 深入了解并行编程：Is Parallel Programming Hard, And, If So, What Can You Do About It?

tips: 由于多核处理器 cpu 之间独立的 L1/L2 cache，会出现 cache line 不一致的问题，为了解决这个问题，有相关协议模型，比如 MESI 协议来保证 cache 数据一致，同时由于 CPU 对「缓存一致性协议」进行的异步优化，对写和读

分别引入了「store buffer」和「invalid queue」，很可能导致后面的指令查不到前面指令的执行结果（各个指令的执行顺序非代码执行顺序），这种现象很多时候被称作「CPU 乱序执行」，为了解决乱序问题（也可以理解为可见性问题，修改完没有及时同步到其他的 CPU），又引出了「内存屏障」的概念；内存屏障可以分为三种类型：写屏障，读屏障以及全能屏障（包含了读写屏障），屏障可以简单理解为：在操作数据的时候，往数据插入一条”特殊的指令”。只要遇到这条指令，那前面的操作都得「完成」。CPU 当发现写屏障指令时，会把该指令「之前」存在于「store Buffer」所有写指令刷入高速缓存。就可以让 CPU 修改的数据马上暴露给其他 CPU，达到「写操作」可见性的效果。读屏障也是类似的：CPU 当发现读屏障的指令时，会把该指令「之前」存在于「invalid queue」所有的指令都处理掉。通过这种方式就可以确保当前 CPU 的缓存状态是准确的，达到「读操作」一定是读取最新的效果。由于不同 CPU 架构的缓存体系不一样、缓存一致性协议不一样、重排序的策略不一样、所提供的内存屏障指令也有差异，所以一些语言 c++/java/go/rust 都有实现自己的内存模型, 比如 golang 大牛 Russ Cox 写的内存模型系列文章 **Memory Models**: https://research.swtch.com/mm

---

License:

- 本书采用Creative Commons Attribution-NonCommercial 4.0 International Public License开源，具体条款可参考 LICENSE 文件。
- 翻译 Copyright © weedge

通告

责任。工程和软件开发领域的知识和最佳实践不断变化。从业者和研究人员必须始终依靠自己的经验和知识来评估和使用此处描述的任何信息、方法、化合物或实验。在使用这些信息或方法时，他们应注意自己的安全和他人的安全，包括对其职业责任的各方。

在法律允许的最大范围内，作者、贡献者或编辑均不对任何因产品责任、疏忽或其他原因而导致的人身或财产伤害承担任何责任，也不对此处所述材料中的任何方法、产品、说明或观念的使用或操作承担责任。

商标。公司用来区分其产品的名称通常被视为商标或注册商标。Intel、Intel Core、Intel Xeon、Intel Pentium、Intel Vtune 和 Intel Advisor 是 Intel Corporation 在美国和/或其他国家的商标。AMD 是 Advanced Micro Devices Corporation 在美国和/或其他国家的商标。ARM 是 Arm Limited（或其子公司）在美国和/或其他地方的商标。然而，读者应联系相应的公司获取有关商标和注册的完整信息。

从属关系。在撰写本书时，主要作者（Denis Bakhvalov）是 Intel Corporation 的雇员。本书中提供的所有信息都不是上述公司的官方立场，而是作者个人的知识和观点。主要作者在撰写本书时没有收到 Intel Corporation 的任何财务赞助。

广告。本书不宣传任何软件、硬件或其他产品。

版权

版权所有 © 2020 Denis Bakhvalov，采用创作共用许可证（CC BY 4.0）。

前言

关于作者

Denis Bakhvalov 是 Intel 的高级开发人员，负责 C++ 编译器项目，旨在为各种不同架构生成优化代码。性能工程和编译器始终是他主要关注的领域之一。Denis 于 2008 年开始作为软件开发人员的职业生涯，并在多个领域工作，包括开发桌面应用程序、嵌入式系统、性能分析和编译器开发。2016 年，Denis 开始了他的 easyperf.net 博客，专注于性能分析和调优、C/C++ 编译器以及 CPU 微体系结构。Denis 是积极生活方式的支持者，在业余时间喜欢踢足球、打网球、跑步和下棋。此外，Denis 还是两个美丽女儿的父亲。

联系方式：

- 电子邮件：dendibakh@gmail.com
- Twitter：@dendibakh
- LinkedIn：@dendibakh

作者的话

我开始撰写这本书的初衷很简单：教育软件开发人员更好地理解其应用程序在现代硬件上的性能。我知道这对于初学者甚至有经验的开发人员来说可能是一个令人困惑的话题。这种困惑主要发生在没有先前处理性能相关任务的开发人员身上。这没关系，因为每个专家都曾经是初学者。

我记得我开始进行性能分析的日子。我盯着陌生的度量标准，试图匹配不符合的数据。我感到困惑。直到最后一切“豁然开朗”，所有的拼图才拼在一起。那时候，唯一的良好信息来源是软件开发人员手册，而这并不是主流开发人员喜欢阅读的。因此，我决定写这本书，希望能让开发人员更容易学习性能分析的概念。

认为自己在性能分析方面是初学者的开发人员可以从本书的开头开始顺序阅读，逐章阅读。第 2 至 4 章为开发人员提供了后续章节所需的最低知识集。已经熟悉这些概念的读者可以选择跳过这些章节。此外，本书还可以用作优化软件应用程序的参考或检查清单。开发人员可以将第 7 至 11 章作为调整其代码思路的来源。

目标受众

本书将主要对那些从事性能关键应用程序和进行低级优化的软件开发人员有用。仅举几个领域：高性能计算（HPC）、游戏开发、数据中心应用程序（如 Facebook、Google 等）、高频交易。但是，本书的范围并不仅限于上述行业。任何想更好地了解其应用程序性能并知道如何进行诊断和改进的开发人员都会从中受益。作者希望本书所呈现的材料能帮助读者培养新的技能，可以应用于他们的日常工作中。

读者预期具备 C/C++ 编程语言的最低背景，以理解本书的示例。能够阅读基本的 x86 汇编语言是理想的，但不是严格要求。作者还期望读者熟悉计算机体系结构和操作系统的基本概念，如中央处理器、内存、进程、线程、虚拟和物理内存、上下文切换等。如果上述术语中有任何新的术语，建议先学习这些材料。

致谢

非常感谢 Mark E. Dawson，他在撰写本书的几个部分时提供了帮助：《为 DTLB 优化》（Section 8.4）、《为 ITLB 优化》（Section 11.8）、《缓存预热》（Section 12.2.2）、系统调优（Section 12.4）、关于多线程应用程序性能扩展和开销的部分（Section 13.1）、使用 COZ 性能分析器的部分（Section 13.5）、关于 eBPF 的部分（Section 13.6）、《检测一致性问题》（Section 13.7）。Mark 是高频交易行业的知名专家。在撰写本书的不同阶段，Mark 非常乐意分享他的专业知识和反馈意见。

接下来，我要感谢 Sridhar Lakshmanamurthy，在关于 CPU 微体系结构的部分（Chapter 3）中撰写了大部分内容。Sridhar 在 Intel 工作了几十年，是半导体行业的老手。

非常感谢 Nadav Rotem，他是 LLVM 编译器中向量化框架的原始作者，他帮助我撰写了关于向量化的部分（Section 9.3.5）。

Clément Grégoire 编写了关于 ISPC 编译器的部分（Section 9.3.7.5）。Clément 在游戏开发行业拥有丰富的背景。他的评论和反馈帮助解决了游戏开发行业中一些挑战。

如果没有以下评论员的帮助，本书不可能从草稿中走出来：Dick Sites、Wojciech Muła、Thomas Dullien、Matt Fleming、Daniel Lemire、Ahmad Yasin、Michele Adduci、Clément Grégoire、Arun S. Kumar、Surya Narayanan、Alex Blewitt、Nadav Rotem、Alexander Yermolovich、Suchakrapani Datt Sharma、Renat Idrisov、Sean Heelan、Jumana Mundichipparakkal、Todd Lipcon、Rajiv Chauhan、Shay Morag 等等。

此外，我还要感谢整个性能社区为无数的博客文章和论文。我通过阅读 Travis Downs、Daniel Lemire、Andi Kleen、Agner Fog、Bruce Dawson、Brendan Gregg 等人的博客，学到了很多东西。我站在巨人的肩膀上，本书的成功不仅归功于我自己。本书是我向整个社区致敬和回馈的方式。

最后但并非最不重要的是，感谢我的家人，他们耐心地容忍我错过周末旅行和晚间散步。没有他们的支持，我不可能完成这本书。

目录

目录 7

1 介绍 13
1.1 为什么我们仍然需要性能调优? . . . 14
1.2 谁需要性能调优? . . . 15
1.3 什么是性能分析? . . . 16
1.4 本书讨论了什么? . . . 17
1.5 本书未涉及的内容 . . . 17
1.6 练习 . . . 18

第一部分：在现代 CPU 上的性能分析 19

2 性能测量 19
2.1 现代系统中的噪声 . . . 19
2.2 在生产环境中进行性能测量 . . . 21
2.3 自动检测性能回归 . . . 21
2.4 手动性能测试 . . . 23
2.5 假设检验方法 . . . 25
2.6 软件和硬件定时器 . . . 25
2.7 微基准测试 . . . 28

3 CPU 微体系结构 31
3.1 指令集架构 . . . 31
3.2 流水线技术 . . . 31
3.3 开发指令级并行性（ILP） . . . 33
3.3.1 乱序执行（OOO Execution） . . . 33
3.3.2 超标量引擎和 VLIW . . . 34
3.3.3 推测执行 . . . 34
3.3.4 分支预测 . . . 35
3.4 SIMD 多处理器 . . . 36
3.5 开发线程级并行性 . . . 37
3.5.1 多核系统 . . . 38
3.5.2 同时多线程 . . . 38
3.5.3 混合架构 . . . 39
3.6 存储器层次结构 . . . 40
3.6.1 缓存层次结构 . . . 40
3.6.1.1 数据在缓存中的放置 . . . 40
3.6.1.2 在缓存中查找数据 . . . 41
3.6.1.3 处理未命中 . . . 41

3.6.1.4 处理写操作 . . . 41
3.6.1.5 其他缓存优化技术 . . . 42
3.6.2 主存储器 . . . 42
3.6.2.1 DDR . . . 42
3.6.2.2 GDDR 和 HBM . . . 45
3.7 虚拟内存 . . . 45
3.7.1 转译后备缓冲区 (TLB) . . . 47
3.7.2 大页 . . . 47
3.8 现代 CPU 设计 . . . 48
3.8.1 CPU 前端 . . . 49
3.8.2 CPU 后端 . . . 49
3.8.3 Load-Store Unit . . . 50
3.8.4 TLB 层次结构 . . . 51
3.9 性能监控单元 . . . 52
3.9.1 性能监控计数器 . . . 53

4 性能分析中的术语和指标 56
4.1 已退役 (Retired) vs. 已执行 (Executed) 指令 . . . 56
4.2 CPU 利用率 . . . 56
4.3 CPI 和 IPC . . . 57
4.4 微操作 . . . 58
4.5 流水线槽 . . . 59
4.6 核心周期与参考周期 . . . 60
4.7 缓存失效 . . . 60
4.8 错误预测的分支 . . . 61
4.9 性能指标 . . . 62
4.10 内存延迟和带宽 . . . 64
4.11 案例研究：分析四个基准测试的性能指标 . . . 67

5 性能分析方法 74
5.1 代码插桩 (Instrumentation) . . . 74
5.2 跟踪 . . . 77
5.3 工作负载特征化 . . . 78
5.3.1 计数性能事件 . . . 79
5.3.2 手动收集性能计数器数据 . . . 79
5.3.3 多路复用和事件缩放 . . . 80
5.4 使用标记器 API . . . 81
5.5 采样 . . . 84
5.5.1 用户模式和基于硬件事件的采样 . . . 85
5.5.2 寻找热点 . . . 85
5.5.3 收集调用堆栈 . . . 87
5.6 Roofline 性能模型 . . . 88
5.7 静态性能分析 . . . 92
5.7.1 案例研究：使用 UICA 优化 FMA 吞吐量 . . . 93
5.8 编译器优化报告 . . . 95

6 CPU 特性用于性能分析 100
6.1 自顶向下微架构分析 . . . 101
6.1.1 在英特尔平台上的 TMA . . . 101
6.1.2 TMA 在 AMD 平台上 . . . 107
6.1.3 TMA 在 ARM 平台上 . . . 108
6.1.4 TMA 总结 . . . 110
6.2 分支记录机制（Branch Recording Mechanisms） . . . 111
6.3 未记录未执行的分支 . . . 112
6.3.1 英特尔平台上的 LBR . . . 112
6.3.2 AMD 平台上的 LBR . . . 114
6.4 ARM 平台上的 BRBE . . . 114
6.4.1 捕获调用堆栈 . . . 114
6.4.2 识别热点分支 . . . 115
6.4.3 分析分支预测错误率 . . . 116
6.4.4 机器码的精确计时 . . . 116
6.4.5 估计分支结果概率 . . . 119
6.4.6 提供编译器反馈数据 . . . 119
6.5 基于硬件的采样功能 . . . 119
6.5.1 英特尔平台上的 PEBS . . . 119
6.5.2 AMD 平台上的 IBS . . . 120
6.5.3 ARM 平台上的 SPE . . . 121
6.5.4 精确事件 . . . 121
6.6 分析内存访问 . . . 122

7 性能分析工具概述 125
7.1 Intel Vtune . . . 125
7.2 AMD uProf . . . 128
7.3 Apple Xcode Instruments . . . 131
7.4 Linux Perf . . . 134
7.5 火焰图 . . . 135
7.6 Windows 事件跟踪 . . . 135
7.6.1 如何配置它 . . . 136
7.6.2 你能用它做什么： . . . 136
7.6.3 你不能用它做什么： . . . 136
7.7 记录 ETW 跟踪的工具 . . . 136
7.7.1 查看和分析 ETW 跟踪的工具 . . . 137
7.7.1.1 设置 . . . 137
7.7.1.2 捕获痕迹 . . . 137
7.7.1.3 在 WPA 中分析 . . . 138
7.8 专业和混合性能分析器 . . . 140
7.9 Tracy 的其他功能 . . . 144
7.10 持续性能分析 . . . 145

第二部分：源代码优化 149

8 优化内存访问 152

8.1 缓存友好数据结构 . . . 152
8.1.1 顺序访问数据 . . . 153
8.1.2 使用合适的容器 . . . 153
8.1.3 数据压缩 . . . 153
8.1.4 对齐和填充 . . . 154
8.1.5 动态内存分配 . . . 155
8.1.6 调整代码以适应内存层次结构 . . . 156
8.2 显式内存预取 . . . 156
8.3 内存分析 . . . 159
8.4 减少 DTLB 未命中 . . . 159
8.4.1 显式大页面 (EHP)。 . . . 160
8.4.2 透明大页面 (THP) . . . 160
8.4.3 显式大页面 (EHP) vs. 透明大页面 (THP) . . . 162
8.5 问题与练习 . . . 162
8.6 章节总结 . . . 163

9 优化计算 164
9.1 数据流依赖 . . . 164
9.2 内联函数 . . . 168
9.3 循环优化 . . . 169
9.3.1 低级优化 . . . 170
9.3.2 高级优化 . . . 171
9.3.3 发现循环优化机会 . . . 173
9.3.4 循环优化框架 . . . 174
9.3.5 向量化 . . . 174
9.3.6 编译器自动向量化 . . . 175
9.3.7 发现向量化机会 . . . 176
9.3.7.1 向量化是非法的 . . . 176
9.3.7.2 向量化无益 . . . 178
9.3.7.3 循环已向量化，但使用标量版本 . . . 179
9.3.7.4 循环以次优方式向量化 . . . 179
9.3.7.5 使用显式向量化的语言 . . . 179
9.4 编译器内联函数 . . . 180
9.4.1 内部函数的包装库 . . . 181

10 优化分支预测 185
10.1 用查找表替换分支 (Replace Branches with Lookup) . . . 185
10.2 用算术替换分支 (Replace Branches with Arithmetic) . . . 186
10.3 用谓词替换分支 (Replace Branches with Predication) . . . 187

11 机器代码布局优化 191
11.1 机器代码布局 . . . 191
11.2 基本块 . . . 192
11.3 基本块布局 . . . 192
11.4 基本块对齐 . . . 194
11.5 函数拆分 . . . 196

11.6 函数重排序 . . . 196
11.7 基于性能分析引导的优化（Profile Guided Optimizations） . . . 197
11.8 减少 ITLB 未命中 . . . 200
11.9 案例研究：测量代码足迹 (Measuring Code Footprint) . . . 202

12 其他优化领域 **206**
12.1 优化输入输出 . . . 206
12.2 低延迟优化技术 . . . 206
12.2.1 避免小页面错误 . . . 207
12.2.2 高速缓存预热 . . . 208
12.2.3 避免 TLB 驱逐 . . . 209
12.2.4 防止意外内核节流 . . . 210
12.3 缓慢的浮点运算 . . . 210
12.4 系统调优 . . . 211
12.5 案例研究：对最后一级缓存大小的敏感性 . . . 212
12.6 AMD Milan 7313P 处理器的集群式内存层次结构 . . . 213
12.6.1 控制和监控 LLC 分配 . . . 214

13 优化多线程应用 **218**
13.1 性能扩展和开销 . . . 218
13.2 并行效率指标 . . . 220
13.2.1 有效 CPU 利用率 . . . 220
13.2.2 线程数 . . . 220
13.2.3 等待时间 . . . 221
13.2.4 自旋时间 . . . 221
13.3 使用 Intel VTune Profiler 进行分析 . . . 221
13.3.1 查找开销大的锁 . . . 222
13.3.2 平台视图 . . . 222
13.4 使用 Linux Perf 进行分析 . . . 224
13.4.1 查找开销大的锁 . . . 225
13.5 使用 Coz 进行分析 . . . 226
13.6 利用 eBPF 和 GAPP 进行分析 . . . 226
13.7 缓存一致性问题 . . . 227
13.7.1 缓存一致性协议 . . . 227
13.7.2 真共享 . . . 229
13.7.3 伪共享 . . . 229

14 软件和硬件性能的当前和未来趋势 **232**
后记 . . . 232

术语表 **234**

主要 CPU 微架构列表 **235**

References **236**

附录 A. 减少测量噪声 (Reducing Measurement Noise) **240**

附录 B. LLVM 向量化 (Vectorizer) 244

附录 C. 启用大页面 248

14.1 Windows 248

14.2 Linux 248

显式大页面 248

附录 D. Intel 处理器跟踪 251